

```

import React, { forwardRef, useEffect, useRef, useCallback } from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface MemoBoxActionConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
  tooltipText?: string;
}

export interface MemoBoxProps extends
  React.TextareaHTMLAttributes<HTMLTextAreaElement> {
  label?: string;
  error?: string;
  actionIcons?: MemoBoxActionConfig[];
}

export const MemoBox = forwardRef<HTMLTextAreaElement, MemoBoxProps>(
  (
    {
      label,
      error,
      actionIcons = [],
      className,
      disabled,
      required,
      placeholder,
      rows = 4,
      ...props
    },
    ref,
  ) => {
    const textareaRef = useRef<HTMLTextAreaElement>(null);
    const hasError = Boolean(error);

    const handleRef = (node: HTMLTextAreaElement | null) => {
      textareaRef.current = node;
      if (ref) {
        if (typeof ref === "function") {
          ref(node);
        } else {
          ref.current = node;
        }
      }
    };
  });

```

```

const adjustHeight = useCallback(() => {
  const textarea = textareaRef.current;
  if (textarea) {
    textarea.style.height = "auto";
    const newHeight = textarea.scrollHeight;
    textarea.style.height = `${newHeight}px`;
  }
}, []);

useEffect(() => {
  adjustHeight();
}, [adjustHeight]);

const handleChange = (e: React.ChangeEvent<HTMLTextAreaElement>) => {
  adjustHeight();
  props.onChange?.(e);
};

const limitedActionIcons = actionIcons.slice(0, 4);

return (
  <div className={cn("w-full", className)}>
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="relative">
      <textarea
        ref={handleRef}
        className={cn(
          "w-full bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-surface",
          "text-text-main placeholder:text-muted-foreground/60",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
          "focus-visible:ring-offset-2 focus-visible:ring-offset-background",
          "disabled:opacity-50 disabled:pointer-events-none",
          disabled:cursor-not-allowed",

```

```

    "transition-all duration-200 ease-out",
    "resize-none",
    "px-4 py-3",
    "min-h-[100px]",
    hasError &&
      "border-destructive/50 focus-visible:ring-destructive/50",
    disabled && "bg-muted/50",
  )}
  disabled={disabled}
  required={required}
  placeholder={placeholder}
  rows={rows}
  onChange={handleChange}
  {...props}
/>
{limitedActionIcons.length > 0 && (
  <div className="absolute bottom-2 right-2 flex items-center gap-1.5">
    {limitedActionIcons.map((action, index) => (
      <button
        key={index}
        type="button"
        className={cn(
          "relative flex items-center justify-center",
          "w-8 h-8 rounded-md",
          "text-muted-foreground/60 hover:text-foreground",
          "bg-transparent hover:bg-accent",
          "transition-all duration-150 ease-out",
          "active:scale-95",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
          "focus-visible:ring-offset-2",
          "disabled:opacity-50 disabled:pointer-events-none",
        )}
        aria-label={action.tooltipText}
        disabled={disabled}
        onClick={action.onClick}
      >
        <action.icon className="w-4 h-4" aria-hidden="true" />
      </button>
    )}}
  </div>
)}
</div>
{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >

```

```
        >
        {error}
    </p>
    )}
</div>
);
},
);
```

```
MemoBox.displayName = "MemoBox";
```